

## ECE 251 Lab 6: Subroutines and the Stack

**Name:** Zach Whitmore

**Lab Day:** Friday, 8 AM

**Date:** 11/03/2025

### Objectives

- The objective of this lab was to gain experience working with advanced assembler directives, conditional branching, and monitor utility subroutines, which included:
  - Performing arithmetic and logical operations on data
  - Using directives to define constants, allocate memory, and organize code
  - Developing and testing a software delay loop
  - Applying conditional branching and looping to control program flow
  - Building and debugging multi-subroutine programs
- **Tools:** VSCode, PlatformIO, Tiva board
- **Language:** ARM assembly

### List of Project files:

1. Part 6a: lab6a.s, setup.s
2. Part 6b: lab6b.s, setup.s
3. Part 6c: lab56.s, ECE215\_asm\_utils.s, setup.s

### Procedure

#### Part 6a: Table ABS and Square

For Part A I was given a short program that used a table of signed byte values and required creating two new tables in memory. One to store the absolute value of each byte and the other to store the square of each value. At first I wasn't completely sure how to handle the absolute value calculation in assembly, since negative numbers needed to be converted to positive while positive numbers stayed the same. I started by comparing the value in R4 to zero and experimenting with different conditional instructions like LT (less than) before realizing that the MI (minus negative) condition flag was the simplest way to keep the positive values and convert the negatives. As by using RSBMI (Reverse sub minus negative) I could subtract zero by the value in the constant table if the value gave a negative PSR bit to get the positive of that value.

Once I had the absolute values stored correctly in the table, the next step was squaring each value. This part was more straightforward, I just multiplied R4 by itself and stored the result as a halfword since the absolute values squared would be equal to the constant values squared. To verify that everything was working correctly I viewed both tables in memory. The results

appeared in hexadecimal, so I converted them to decimal to confirm the absolute values and squares matched my expectations.

### Output:

| Memory [0x20000000+10].dbgmem |                                                         |  |  |  |  |  |  |  |  |  |  |  |  |  |  | Memory [0x2000000a+20].dbgmem |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|-------------------------------|---------------------------------------------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|-------------------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
| 1                             | Offset: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                               |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 2                             | 20000000: 0A 2D 3F 0C 2B 38 20 2D 62 01                 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                               |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

Figure 1. Part 6a: Absolute values in memory

| Memory [0x20000000+10].dbgmem |                                                         |  |  |  |  |  |  |  |  |  |  |  |  |  |  | Memory [0x2000000a+20].dbgmem X |                   |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|-------------------------------|---------------------------------------------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|---------------------------------|-------------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
| 1                             | Offset: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                 |                   |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 2                             | 20000000:                                               |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                 | 64 00 E9 07 81 0F |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 3                             | 20000010: 90 00 39 07 40 0C 00 04 E9 07 84 25 01 00     |  |  |  |  |  |  |  |  |  |  |  |  |  |  |                                 |                   |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

Figure 2. Part 6a: Squared values in memory

## Procedure

### Part 6b:

For Part B I was asked to write a delay subroutine that would pause the program for roughly one second using a simple software loop. To do this I needed to determine how many loop iterations were required based on the processor's clock cycle time. Since the Tiva board runs at 16 MHz, each clock cycle takes about 62.5 nanoseconds. The loop uses a SUBS instruction followed by a BNE, which together take three cycles per iteration. By dividing one second by 3 times 62.5 nanoseconds, I calculated a value for the delay of 5,333,333 and defined Delay\_Clocks in the program as this value.

I then set up a main loop that called the delay subroutine ten times to create a total delay of about ten seconds. To test it, I used a stopwatch on my phone to measure the total runtime from the start to the end of the loop. The total time came out to about 10.3 seconds, which was close enough that any small difference was likely due to my reaction time when starting and stopping the timer.

## Procedure

### Part 6c: The Elevator

For Part C I was asked to create a program that simulates a simple elevator system by writing multiple subroutines that handle user input, output messages, timing delays, and floor movement. The purpose of this part was to learn how to structure a larger assembly program

using subroutines, register preservation, and stack management while integrating input and output through the provided utility class.

I began by writing the `OpenDoor` subroutine. This routine first calls the given `Delay` subroutine, which I set to a one second delay by reusing the `DELAY_CLOCKS` value from Part B to simulate the time it takes for the doors to open. After the delay finishes, I load the string constant I defined as `OpenMSG` from memory into `R0` and call `OutStr` from the provided utility class to print “Door Open” to the terminal. I used `PUSH {LR}` at the start and `POP {LR}` before returning to ensure the link register with the return address was preserved.

Next, I wrote the `CloseDoor` subroutine using the same structure as `OpenDoor`. It also calls `Delay`, loads the `CloseMSG` string into `R0`, and uses `OutStr` to print “Door Close” to the terminal. Both of these routines provided a consistent way to show door movement with a visible one second delay between each action.

After completing the door routines, I moved on to the elevator movement logic. I started with the `Up` subroutine, which begins by printing the message “Going UP” to the terminal using `OutStr`. I then loaded the current floor value from memory into `R5` and the user selected the destination floor into `R6`. Inside a loop, the current floor value is incremented by one, and then `Delay` is called to simulate travel time between floors. After each delay, I use `OutHex` to print the current floor number in hexadecimal, followed by a space character printed with `OutChar`. Once the loop reaches the destination floor I stored the updated floor number back into the `currentFloor` variable in memory and returned to the main program.

The `Down` subroutine was written similarly, except it decrements the current floor value instead of incrementing it. It starts by printing “Going DOWN,” then uses the same delay and display process as `Up`, calling `Delay`, `OutHex`, and `OutChar` on each iteration until it reaches the requested floor. When finished, it stores the new floor value in `currentFloor` and returns.

I then wrote the `Stay` subroutine for cases where the user selects the same floor they are already on. This routine simply calls `Delay` and prints “Same Floor” using `OutStr` before returning to the main loop.

With the individual routines written, I created the main program. At the start of main, I initialized `currentFloor` to 1 and called `OpenDoor` to display the initial state. The program then used `InChar_Echo` to read a single ASCII digit entered by the user and automatically echo it to the terminal. Since the character input was ASCII I subtracted `0x30` to convert it into a numeric value between 0 and 9 and stored that value in `R4`. I then called `CloseDoor`, compared the user input to the current floor value, and branched to either `Up`, `Down`, or `Stay` depending on the result.

During testing the program mostly worked as intended, but I ran into several issues that required troubleshooting. At first, the elevator wouldn't always reach the correct floor because the floor value was being lost between subroutine calls. This was caused by not properly preserving registers that held the current and desired floor values. I also had a problem where the program never exited the Up or Down loops due to incorrect branch conditions, which caused the program to count past the desired floor. After reviewing my compare and branch instructions, I corrected the logic so the loop exited once the target floor was reached. I also had to fix some spacing problems in the terminal output where the floor numbers and messages would run together, which I solved by adding a space character output using OutChar after each printed floor number. Within the main program, I made a few additional changes to improve stability: I reloaded the user input value back into R0 several times to ensure it wasn't lost or overwritten between subroutine calls, and I added small orchestrator subroutines that acted as structured call points for the larger routines like OpenDoor, CloseDoor, Up, and Down. These helped organize the program flow and made debugging much easier. Once these issues were resolved, the terminal correctly displayed "Door Open," waited for the floor input, then printed "Door Close," followed by "Going UP" or "Going DOWN" with the floor numbers updating smoothly. When the same floor was entered, the program displayed "Same Floor" as expected.

To prevent the register corruption that caused my early logic errors, I updated the Up and Down routines to use PUSH/POP {R4, R5, R6, LR} at the beginning and end of each subroutine. This ensured that my working registers and the link register were safely preserved while calling monitor utilities like OutStr and OutHex, which use several of the lower registers internally. Before making that change, values stored in R0 through R3 were often overwritten, causing unpredictable behavior. By switching to higher registers and explicitly saving them on the stack, I eliminated data loss and made the overall program much more stable and reliable.

### Output:

```
Door Open
9Door Close
Going UP
2 3 4 5 6 7 8 9 Door Open
1Door Close
Going DOWN
8 7 6 5 4 3 2 1 Door Open
1Door Close
Same Floor
Door Open
█
```

Figure 3. Part 6c: Terminal with up, down, and same movements

## Questions:

**QUESTION A:** How do we return a value? Return an array of values?

**Ans:** To return a value from a subroutine place you need to put it in a register before using BX LR. If you want to return an array of values you should return the starting address of the data into a register instead.

**QUESTION 1:** Consider the program written in Figure 6.4 and the memory map in Figure 6.5. Manually calculate the values of the memory map after the instructions on lines 8, 9, 10, 11, and 12 are executed. In other words, you should draw five separate memory maps and show the memory contents after each instruction. Assume the SP is initialized to 0x2000.0400 and that the stack contents are initialized to 0x00.

**Ans:** See Memory maps at end of report

**QUESTION 2:** When you use push and pop instructions, why do they have to be paired?

**Ans:** If you push a value onto the stack but don't pop it the value stays on the stack and the stack pointer isn't restored to its original position. This causes the stack to become unbalanced which can lead to incorrect data being read or the program returning to the wrong address.

**QUESTION 3:** What does the following bit of code do?

```
1    PUSH {R0}
2    PUSH {R1}
3    POP {R0}
4    POP {R1}
```

Do you think this is a good way to do this, or is there a better way?

**Ans:** The first PUSH {R0} stores R0 on the stack, and the second PUSH {R1} stores R1 on top of it. When you then POP {R0}, it loads the value from R1 into R0, and the next POP {R1} loads the value that was originally in R0 into R1. This effectively swaps the two register values. That's fine if your goal is to exchange their contents, but if you were trying to save and restore them such as storing an address or variable for later use this method would not work correctly.

**QUESTION 4:** Regarding the previous question, what if you coded this as:

```
1    PUSH {R0,R1}
2    POP {R0,R1}
```

Would you get the same results as the four instructions above? Why or why not?

**Ans:** When you use PUSH {R0, R1} and POP {R0, R1}, it doesn't behave the same as pushing and popping each register individually. In the grouped form, both registers are pushed onto the stack together with the lowest register (R0) ending up on top of the stack. When they are popped, the lowest register is restored first, so each register gets back its original value. In contrast, doing two separate pushes and pops reverses the order, which causes the values in R0 and R1 to swap. Using the combined form is the correct way to save and restore multiple registers in a subroutine because it preserves the original data.

### Question 1 Memory Maps:

Memory Map 1. Line 8

|             |       |            |
|-------------|-------|------------|
| 0x2000.03F8 | 00    | SP         |
| 0x2000.03F9 | 00    | 0x200003FC |
| 0x2000.03FA | 00    | PC         |
| 0x2000.03FB | 00    |            |
| 0x2000.03FC | 42 SP | R0         |
| 0x2000.03FD | 00    | 0x00000042 |
| 0x2000.03FE | 00    |            |
| 0x2000.03FF | 00    | R1         |
| 0x2000.0400 | 00    | 0x00000055 |

Memory Map 2. Line 9

|             |       |            |
|-------------|-------|------------|
| 0x2000.03F8 | 55 SP | SP         |
| 0x2000.03F9 | 00    | 0x200003F8 |
| 0x2000.03FA | 00    | PC         |
| 0x2000.03FB | 00    |            |
| 0x2000.03FC | 42    | R0         |
| 0x2000.03FD | 00    | 0x00000042 |
| 0x2000.03FE | 00    |            |
| 0x2000.03FF | 00    | R1         |
| 0x2000.0400 | 00    | 0x00000055 |

Memory Map 3. Line 10

|             |       |            |
|-------------|-------|------------|
| 0x2000.03F8 | 55 SP | SP         |
| 0x2000.03F9 | 00    | 0x200003F8 |
| 0x2000.03FA | 00    | PC         |
| 0x2000.03FB | 00    |            |
| 0x2000.03FC | 42    | R0         |
| 0x2000.03FD | 00    | 0x00000042 |
| 0x2000.03FE | 00    |            |
| 0x2000.03FF | 00    | R1         |
| 0x2000.0400 | 00    | 0x00000055 |

Memory Map 4. Line 11

|             |       |            |
|-------------|-------|------------|
| 0x2000.03F8 | 55    | SP         |
| 0x2000.03F9 | 00    | 0x200003FC |
| 0x2000.03FA | 00    | PC         |
| 0x2000.03FB | 00    |            |
| 0x2000.03FC | 42 SP | R0         |
| 0x2000.03FD | 00    | 0x00000042 |
| 0x2000.03FE | 00    |            |
| 0x2000.03FF | 00    | R1         |
| 0x2000.0400 | 00    | 0x00000055 |

Memory Map 5. Line 12

|             |       |            |
|-------------|-------|------------|
| 0x2000.03F8 | 55    | SP         |
| 0x2000.03F9 | 00    | 0x20000400 |
| 0x2000.03FA | 00    | PC         |
| 0x2000.03FB | 00    |            |
| 0x2000.03FC | 42    | R0         |
| 0x2000.03FD | 00    | 0x00000042 |
| 0x2000.03FE | 00    |            |
| 0x2000.03FF | 00    | R1         |
| 0x2000.0400 | 00 SP | 0x00000055 |